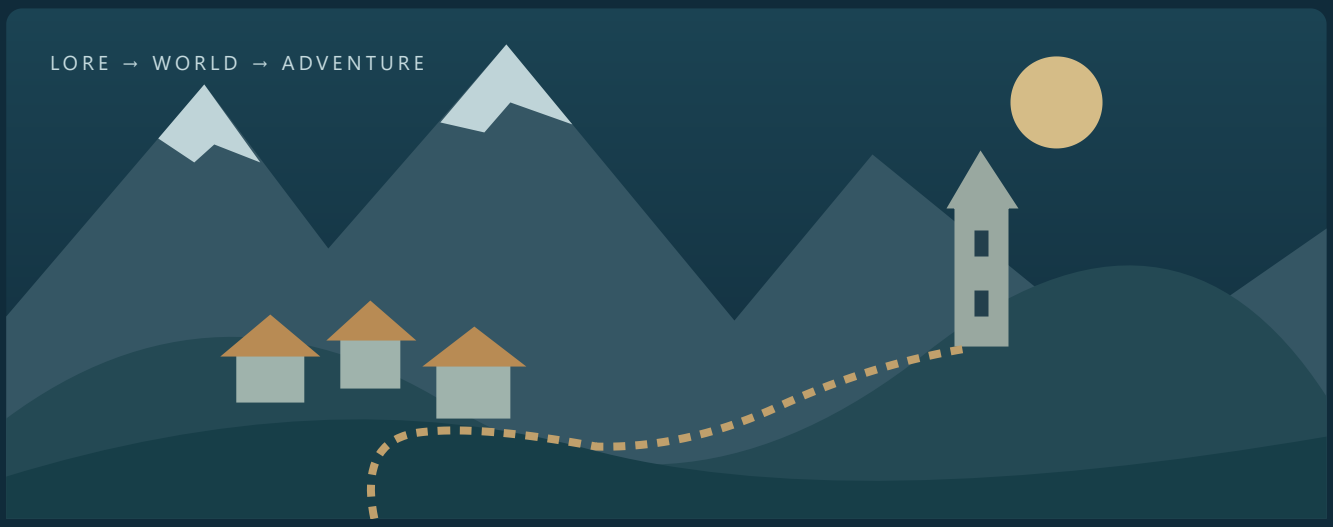
WORKING TITLE • LOREFORGE

Your story.
A world to survive.

A first-person fantasy RPG generated from player-written lore.

A practical plan for exploration, story, survival, crafting, and combat in Three.js — with realistic art direction and local AI as the preferred starting point.

FIRST PERSON | FANTASY | REALISTIC | LOCAL AI FIRST



Prepared for the project owner
10 November 2026

Reference hardware
RTX 4060 · Intel i7-13700H

The game we are planning

The player describes a fantasy world. The game turns that description into a coherent region with terrain, settlements, characters, conflict, and a playable adventure.

> "A forest kingdom is dying from a spreading curse. A mountain village survives by mining glowing crystals, but the old watchtower has started calling people at night."

This input should influence the landscape, architecture, characters, resources, enemies, and main quest. The player then explores in first person, gathers supplies, crafts useful equipment, fights threats, and makes story choices.

Confirmed preferences

Decision	Confirmed direction
Genre	Exploration and story mixed with survival, crafting, and combat RPG systems.
Perspective / setting	First person; fantasy.
Graphics	Realistic art direction: believable scale, materials, lighting, architecture, and animation.
AI budget	Prefer free local inference; consider a paid provider only if local results or speed are insufficient.
Known hardware	NVIDIA RTX 4060 and Intel i7-13700H. System RAM, actual available VRAM, laptop power limits, and storage remain unconfirmed.

> **Recommended approach:** AI designs a structured world specification. Seeded procedural systems build it using a curated asset library. Game code controls movement, combat, economy, and quest state.

First release assumption: a single-player desktop-browser prototype, initially tested on Windows. This is a proposed delivery target, not a confirmed platform requirement.

"Loreforge" is a temporary project name. This document is a design and implementation proposal; no engine, benchmark, or generated game is claimed to exist yet. All schedules and budgets are estimates.

From concept to a tested prototype

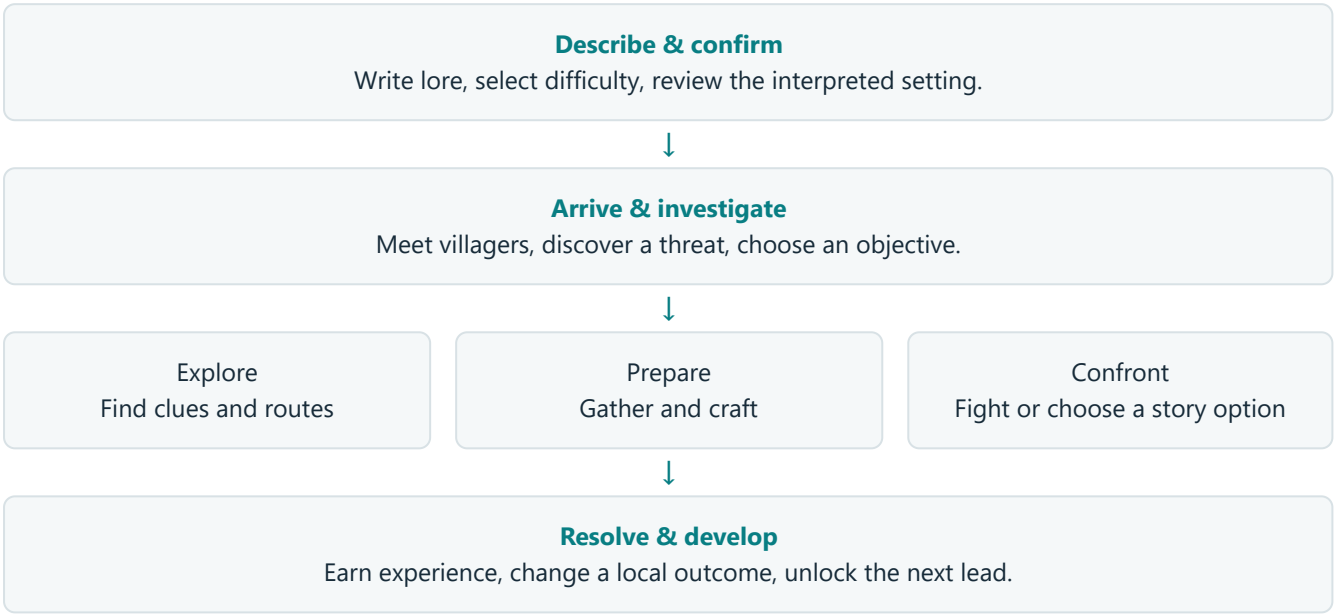
The plan separates the intended player experience, implementation contracts, and evidence needed to decide whether the game works.

Player experience and gameplay loop	04
Prototype scope and expansion boundaries	05
System architecture and technology	06
Lore-to-world generation pipeline	07
Terrain, settlements, and buildings	08
Realistic graphics and asset production	09
NPCs, dialogue, story, and quests	10
Combat and RPG progression	11
Survival, inventory, and crafting	12
Local AI, provider choices, and hardware	13
World data contracts and example specification	14
Code structure, engine loop, and API boundary	15
Save/load, reproducibility, and recovery	16
Screens, controls, and player feedback	17
Development roadmap and milestone gates	18
Testing, generation evaluation, and acceptance	19
Profiling, optimization, and performance targets	20
Budget, risks, and scope decisions	21
Example adventure and implementation kickoff	22
References and glossary	23

> **Reading guide:** the first playable version is deliberately small. Combat, survival, and story all appear, but each starts with a narrow, testable implementation. Broader worlds follow only after the complete loop works.

One world, several reasons to explore

Exploration finds the story. Survival creates preparation. Combat creates risk. Crafting and progression make the next journey possible.



Target first-session experience

1. **Opening:** the player enters a paragraph of lore. A short world summary makes the interpretation visible before generation.
2. **Arrival:** spawn near a safe settlement with basic equipment, supplies, and a clear landmark.
3. **Discovery:** a villager introduces the local problem. A journal records the objective without forcing a long conversation.
4. **Preparation:** gather a few resources and craft one useful item, such as a healing draught.
5. **Expedition:** travel to a ruin, encounter enemies, find evidence, and make one meaningful decision.
6. **Return:** claim the result, see a persistent consequence, and save the adventure.

Design rules

Provide visible goals while allowing detours. Keep survival forgiving by default. Make fights readable through animation and sound. Make story consequences small enough to represent reliably: a changed relationship, an opened route, or a different settlement condition.

Proposed first-session playtime: 20–30 minutes once generated. This is a design target to evaluate with playtests, not a measured result.

A complete small region first

Use a compact vertical slice: a short game that contains every major system and can be played from beginning to end. Preserve realistic art direction while limiting content quantity.

Feature	First playable target	Later expansion
World	One bounded 256 × 256 metre region; forest and foothill variations.	Connected regions, additional biomes, streaming.
Settlement	8–12 modular buildings; two enterable interiors.	Larger towns, more interior templates.
Landmarks	One ruin or tower and one resource site.	Dungeons, caves, faction territories.
Characters	Six named friendly NPCs; two enemy archetypes with bounded encounter counts.	More factions, schedules, creatures.
Story	One main quest with 3–5 steps and two supported endings.	Linked quests and lasting regional stories.
Combat	Sword attack, block, stamina, damage, death, and a basic enemy controller.	Bow, spells, equipment variety, bosses.
Survival / crafting	Health, stamina, hunger; 4 resources and 5 recipes.	Weather exposure, farming, construction.
RPG	Experience and three short upgrade paths.	Broader builds, factions, reputation systems.
AI	World plan, NPC profiles, short contextual dialogue; offline scripted fallback.	Richer memory and new regional planning.

Explicitly deferred

Multiplayer, infinite terrain, player-built castles, fully destructible buildings, arbitrary generated 3D meshes, unrestricted magic systems, voice synthesis, and cinematic facial animation.

> **Prompt support has boundaries.** The first version interprets lore within a supported medieval-fantasy kit. An unsupported request is explained and mapped to the nearest supported option before generation. Custom names, histories, and relationships can vary much more widely than geometry.

Why: realistic graphics require cohesive assets and animation. AI-assisted programming can accelerate scaffolding, but movement, combat feel, navigation, and art integration still need iteration and human testing.

Three layers with clear responsibilities

Browser game / TypeScript + Three.js
Renderer · player controller · physics · combat · inventory · UI · local saves

↕ validated data and events

Local backend / Node.js + TypeScript
Generation jobs · schema validation · world rules · dialogue context · request limits

↕ provider adapter

Local AI
Ollama + tested model

Optional cloud
Explicitly enabled provider

Fallback
Templates + fixed seed

Part	Proposed choice and purpose
Client build	Vite and TypeScript. HTML/CSS UI layered above the canvas; no UI framework required initially.
Rendering / assets	Three.js, WebGLRenderer initially, glTF/GLB models, a managed material and asset cache.
Physics	Rapier; capsule character controller, simple colliders, fixed simulation step. Use a pinned package version.
Procedural generation	Seeded noise, terrain chunks, road graph, placement constraints, modular building grammar; heavy calculations in a worker.
Validation / backend	Shared Zod schemas, a small HTTP server, one generation queue. No remote deployment needed for local development.
Storage / authoring	IndexedDB and JSON export for saves; Blender for preparing assets; Git for source and versioned specifications.

Authority: the local single-player simulation owns world facts and gameplay state. The model proposes descriptions and allowed intents. Neither lore nor AI output becomes executable JavaScript.

Technical basis: Three.js supports glTF loading and Rapier integration. Asset loading and physics remain separate from game-specific systems. [1] [2]

Turn lore into a verified world

1. **Interpret the request.** Extract setting, climate, history, factions, central conflict, mood, and forbidden or required elements. Apply input-length and feature limits.
2. **Compile a world specification.** Ask for a compact structured plan using supported biome, building, enemy, and quest identifiers. Let code assign stable IDs and the seed.
3. **Validate and confirm.** Check schema, counts, references, contradictions, and budget. Show a short summary and any unsupported elements to the player.
4. **Build geography.** Generate terrain and buildable zones. Place settlement and quest landmarks under slope, water, spacing, and travel constraints.
5. **Connect and populate.** Route roads; place modular houses, resource nodes, vegetation, NPC homes, and enemy encounters.
6. **Ground the story.** Bind quest roles to locations and NPCs that actually exist. Generate character details and dialogue context using those facts.
7. **Run playability checks.** Verify safe spawn, door clearance, traversable routes, obtainable quest items, reachable resources, and achievable quest endings.
8. **Publish a playable snapshot.** Save the validated manifest, load essential assets, then enable player control. Stream decorative content afterward if needed.

Bounded recovery

Allow at most one AI repair request for invalid planning data, then use a compatible template. For geometry failures, try at most three deterministic placement variants. If these fail, use a tested layout with the accepted lore and tell the player which elements were simplified.

> **Reproducibility:** retain the accepted AI output and resolved layout. The same seed does not guarantee the same model response. Freeze a generated world instead of asking the model to recreate it on every load.

Expose stages and cancellation in the loading screen. Cancellation discards incomplete jobs; it must not replace an existing valid save.

Geometry follows rules

Terrain and navigation

Divide the initial region into sixteen 64 × 64 metre chunks. Evaluate terrain noise in world coordinates so borders match. Combine broad elevation, smaller surface variation, and biome masks. Start with one terrain style and add variation only after traversability works.

Generate a coarse walkability grid from slope, water, and colliders. Place the village on low-slope land and flatten only its reserved pads. Use A* for road candidates and NPC routes, then validate them against final collision geometry. Add clearance for the actual player capsule, not only a point on a map.

Settlement layout

1. Reserve a central square, access road, and two story locations.
2. Choose plots with enough room for buildings, doors, paths, and small yards.
3. Assign roles: inn, smithy, houses, storage, and quest-related structures.
4. Build the road network and ensure every active entrance connects to it.
5. Add props and vegetation only outside reserved movement corridors.

Building grammar

```
Building = foundation + footprint + wall modules + roof
         + door/window sockets + role-specific props

Style kit: stone base / timber walls / slate or thatch roof
Parameters: width, depth, floors, roof pitch, wear, palette
Constraints: legal sockets, matching scale, clear entrance
Interiors: choose compatible room template and portal
```

AI selects a building role and approved style; the generator picks compatible parts. This allows convincing houses with predictable colliders and reusable materials. Keep two tested interior templates for the first version; other buildings remain exterior-only and communicate that clearly.

Required checks

Reject overlapping footprints, floating foundations, buried entrances, excessive slopes, blocked roads, and quest locations outside the reachable region. Spawn enemies away from the starting safe area and keep their patrols within connected navigation zones.

> **Later:** rivers, caves, bridges, and multi-floor procedural interiors add separate navigation problems. Add them one at a time with dedicated connectivity checks.

Believable fantasy at browser scale

Keep realistic proportions, surface response, lighting, and animation. Limit the area and asset variety so quality stays consistent.

Asset strategy

Build one coherent medieval kit: stone and timber architecture, forest vegetation, worn metal weapons, cloth clothing, and a small set of creatures. Use curated assets and authored modular parts. Runtime text-to-3D generation is a later experiment because it does not provide predictable topology, rigging, colliders, or style.

Asset group	Initial production inventory
Environment	3 tree variants, 3 rocks, grass clusters, bushes, a small ruin kit, and terrain materials.
Buildings	One compatible wall/roof/foundation kit; 4 exterior variations and 2 interior templates.
Characters	One compatible humanoid rig, clothing variations, and 2 enemy presentations. Six profiles can share the rig.
Animation	Idle, walk, run, attack, block/hit response, and death; separate first-person hands/weapon setup.
Items / sound	Sword, shield, resource icons and pickups, crafting props, footsteps, impacts, and ambient loops.

Rendering plan

Use physically based metallic/roughness materials, correct texture color spaces, normal maps, environment lighting, a restrained sun shadow, distance fog, and consistent tone mapping. Three.js MeshStandardMaterial provides this material workflow. [3]

Export optimized GLB assets and prepare low-detail versions of larger objects. Test KTX2/Basis textures and supported geometry compression after the uncompressed pipeline works. KTX2 can reduce delivery size and GPU texture memory through supported compressed formats. [1] [4]

Free asset starting point

Poly Haven offers HDRIs, textures, and models under CC0. It can supply environment materials and lighting; it is not assumed to provide a complete animated fantasy character set. Maintain an asset register with source, license, creator, modifications, and file version for every imported asset. [5]

> **Visual acceptance:** create one reference scene first: a house, path, trees, NPC, and weapon under target lighting. Approve its appearance and performance before multiplying it across the map. Realistic does not imply a large AAA world in the prototype.

Characters remember a consistent world

Generate profiles from the accepted world history: name, role, personality, affiliation, home, goals, relationships, and permitted knowledge. Use a bounded wardrobe and appearance palette so every profile has a valid visual representation.

Simulation layer

Finite states: idle, travel, work, talk, flee, and return home. Navigation and time-of-day rules choose actions. Distant NPCs update less often.

Language layer

AI writes dialogue using relevant lore, character beliefs, recent conversation, and current quest facts. A scripted response works if AI is unavailable.

Knowledge and memory

Keep canonical world facts separate from NPC beliefs. A fearful villager may be wrong about the curse, but the system must label that as a belief. Supply only relevant facts and events to each conversation. Store a short conversation summary plus structured facts such as “player returned the journal.”

Quests as verified state machines

```
Available → Accepted → Investigating → Objective complete
          → Choice resolved → Reward claimed

Each step defines:
preconditions / accepted events / target IDs / completion rule
rewards / failure or recovery route / journal text
```

Start with authored quest structures such as investigate, retrieve, escort-later, and defeat. The AI supplies names, motives, clues, and supported alternatives. A quest compiler binds roles to real NPCs, items, and reachable sites, and checks that required items can be obtained.

Dialogue cannot bypass gameplay rules

Use a response contract containing display text and optional allowed intent IDs. A gameplay handler checks eligibility before accepting a quest, granting a reward, or updating a relationship. Reject unknown IDs and stale state versions. Do not execute model-produced scripts.

Story continuity

Keep quest-critical NPCs safe in the first prototype. If a required item is lost, provide a recoverable route. Handle each reward once. A late dialogue response is ignored if the player has left, combat has begun, or the conversation state has changed.

> **First story goal:** one mystery, six characters with distinct perspectives, one short quest chain, and two endings that produce a visible saved change.

Readable action before a large skill tree

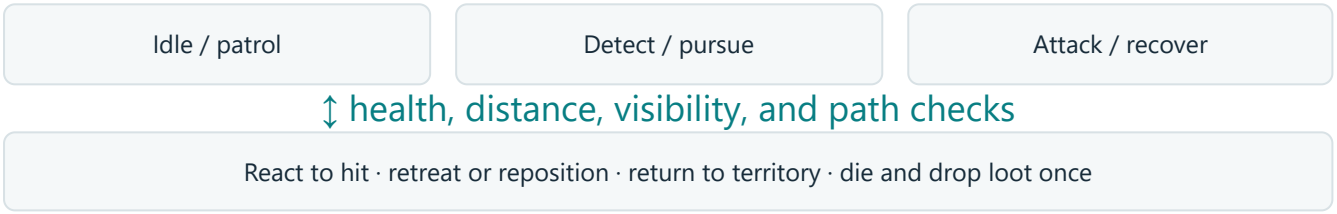
Start with one satisfying weapon and two reliable enemies. More weapon types help only after input, animation, impact, and enemy reactions feel good.

Player combat

Implement sword attacks with wind-up, active hit, and recovery phases. Sample a swept weapon volume during the active window to reduce missed hits at low frame rates. Deduplicate targets so one swing does not apply damage every frame. Blocking has an angle limit and consumes stamina; exhausted players cannot block indefinitely.

Track health, stamina, attack cooldown, equipment, invulnerability windows if used, and hit reactions. Resolve damage in the simulation, using explicit values from equipment data. Never ask an LLM to judge hits or damage during combat.

Enemy behavior



Archetype A: a humanoid melee guard with a slow, telegraphed strike. **Archetype B:** a creature with a short rush and longer recovery. Use the same underlying state machine where possible, with different timing and movement data.

Progression and loot

Path	Initial upgrades
Fighter	Small stamina efficiency and block improvements.
Survivor	Better gathering yield or slower hunger drain.
Explorer	Improved clue visibility or additional contextual dialogue options.

Use a few levels, capped modifiers, and fixed reward tables. Generated flavor can describe a sword, but damage and rarity must stay inside designed ranges. Progression cannot remove the only viable route to a quest.

Death and difficulty

Proposed default: reload a recent checkpoint with a clear explanation. Add harsher survival penalties later. Include an exploration-friendly difficulty that reduces enemy damage and hunger pressure without removing the story.

Bow combat, spellcasting, complex armor, and bosses are expansion features. This plan leaves data hooks for them without making them prototype dependencies.

Preparation supports the adventure

Survival should encourage travel planning without turning every conversation into an urgent search for food. Use gentle hunger in normal difficulty; defer thirst, body temperature, disease, and equipment durability.

Initial resources and recipes

Resource	Source / purpose
Wood	Gathered from marked fallen branches; used for torches and the camp kit.
Stone	Collected from designated deposits; used in basic tools and the camp kit.
Herbs	Visible plants outside the safe area; used in healing and food preparation.
Iron scraps	Loot or deposits near the ruin; used in tools and weapon improvement.

Five starting recipes: torch, healing poultice, simple hatchet, camp kit, and sword upgrade. Ingredient amounts are fixed in recipe data. Bought or looted food covers hunger; a broader cooking system comes later.

Inventory and crafting rules

- Use item IDs, explicit stack sizes, equipment slots, and a simple slot limit.
- Show ingredients owned, ingredients missing, station requirements, and the result before crafting.
- Validate ingredients and output capacity before consuming anything. Apply the transaction once.
- Keep quest items in protected slots so inventory pressure cannot destroy progress.
- Use a smithing station for the sword upgrade. Portable recipes work from the inventory.

Resource placement and recovery

Guarantee that the required recipe resources exist in reachable locations before a quest needs them. Reserve some near the village, with higher-risk supplies farther away. Persist harvested nodes. If regeneration is enabled, store timers and enforce caps rather than rerolling on every load.

Camping

The first camp kit is a simple interaction at approved clear sites, not a freeform construction system. It can create a checkpoint and restore limited stamina. Avoid unrestricted placement near enemies, in doors, or on inaccessible terrain.

> **Balance gate:** a new player should be able to reach the first objective with starter supplies. Crafting offers useful preparation; it should not force repetitive gathering before the game demonstrates its story.

Exact hunger rates, item quantities, damage values, and upgrade costs belong in editable balance tables and will be set through playtests.

Start local, keep providers replaceable

Preferred setup: Ollama running locally behind the game backend. Its local API supports JSON-schema-constrained output. Its cloud service currently does not support that feature, so local and cloud adapters need separate capability checks. [6]

Mode	Use and trade-off
Local AI — preferred	No per-request provider bill for local inference; model download, hardware, electricity, memory, and license terms still matter. Suitable for private single-PC development.
Templates — always available	No AI service required. Tested layouts, NPC responses, and quest templates keep the game playable during failures and benchmarking.
Hosted AI — optional	Useful if local generation is too slow or incoherent. Requires a provider account, explicit enablement, request caps, and a cost limit. Never silently switch to billing.

Your RTX 4060 + i7-13700H

Benchmark a small quantized instruction model in roughly the 3B–8B class, selecting the exact model only after checking installed RAM, available VRAM, license, and structured-output quality. This range is a starting experiment, not a compatibility guarantee. Context length and quantization can change memory use significantly.

The game and local inference compete for memory and compute. Generate the initial plan before the full scene is loaded; then unload the model if required. During play, serialize short dialogue requests and test CPU/offloaded inference against reduced graphics. If that is too slow, use pre-generated dialogue, a smaller model, or an explicitly enabled cloud adapter. Ollama documents memory placement and model unloading controls. [7]

Request policy

Budget one compact world-plan request and one batched character/story request, with at most one repair per failed stage. Dialogue is on demand, one request at a time, with bounded history and output length. Cache stable content; do not run language-model calls in the frame loop.

Failure behavior: timeout, malformed data, unavailable model, or cancellation returns a clear status and a scripted option. Preserve the accepted world and the player's progress.

> **Cost distinction:** local execution and hosted services have different terms. Do not assume any advertised cloud free tier is permanent or enough for a game. Recheck provider pricing before enabling it. [8]

A world description the engine can trust

Separate creative input from resolved simulation data. The model returns semantic choices; the compiler assigns IDs, calculates layout, binds assets, and validates dependencies.

Record	Required responsibilities
WorldSpec	Schema version, lore summary, supported biome/style, history, faction roles, requested counts, conflict, and content limits.
WorldManifest	World ID, seed, generator/asset versions, terrain parameters, resolved transforms, colliders, navigation, spawns, and stable entity IDs.
NPCProfile / QuestSpec	Character facts, beliefs, knowledge tags, home ID; quest steps, preconditions, referenced IDs, rewards, and recovery rules.
SaveGame / DialogueReply	Player/world changes and quest states; dialogue text, allowed intent, request ID, and state version.

Illustrative compiled specification excerpt

```
{
  "schemaVersion": 1,
  "worldId": "world_001",
  "seed": 48172,
  "generatorVersion": "0.1.0",
  "theme": "medieval_fantasy",
  "biome": "temperate_forest",
  "regionMeters": 256,
  "settlement": {"id": "village_01", "buildings": 10},
  "landmarks": [{"id": "tower_01", "kit": "ruined_tower"}],
  "npcCount": 6,
  "conflict": "The tower corrupts the village crystal supply",
  "questTemplate": "investigate_retrieve_choose"
}
```

This excerpt explains the structure; it is not a complete JSON schema or complete save. Resolved geometry, NPCs, quests, and asset bindings are separate manifest records.

Validation beyond JSON syntax

Check enum membership, numeric ranges, unique IDs, reference integrity, and feature support. Then check meaning: a smith has a valid workplace; a quest item exists before its use; resources are reachable; both endings can resolve. A schema alone cannot guarantee those conditions.

Versioning: version schemas, generation algorithms, asset packs, and prompts independently. Store model identity and parameters for provenance, but treat the accepted output as the source of truth.

A small framework with replaceable systems

```
apps/client/src/
  engine/       loop, renderer, input, assets, physics
  world/        chunks, terrain, buildings, navigation
  gameplay/     player, combat, npc, quests, inventory, crafting
  ui/           lore entry, loading, HUD, dialogue, journal
  persistence/  save store, migrations, export
apps/server/src/
  ai/           provider interface, local adapter, fallback
  generation/   jobs, planner, compiler, validation
  dialogue/     context selection, response validation
packages/shared/
  schemas/      contracts and versioned validation
  data/         items, recipes, enemies, supported asset kits
assets/         approved models, textures, animation, licenses
tests/          fixtures, generator checks, browser smoke tests
```

Simulation loop

Read input; advance movement, physics, combat, and gameplay at a fixed step; interpolate visual transforms; update UI; render. Clamp accumulated time and cap catch-up steps so returning from a paused browser tab does not simulate an uncontrolled burst. Pause or explicitly handle hidden-tab time.

Start with modules and stable entity IDs rather than a large generic ECS. Maintain explicit systems and typed events such as **EnemyDefeated**, **ItemCollected**, and **QuestAdvanced**. Add a more complex entity framework only when measured complexity justifies it.

Local backend contract

Route — proposed	Behavior
POST /api/world-jobs	Validate lore/settings, enqueue one bounded job, return job ID.
GET /api/world-jobs/:id	Return stage, warnings, progress estimate, and manifest when ready.
DELETE /api/world-jobs/:id	Cancel work and discard incomplete results.
POST /api/dialogue	Validate NPC/quest snapshot, construct context, return a bounded reply.

Keep cloud secrets on the backend. Bind the development service to localhost; restrict browser origins and request sizes. The backend should derive or validate canonical facts rather than accept arbitrary client-authored instructions as trusted dialogue context.

A generated world should stay generated

Loading a game must restore its accepted world and consequences. It should not ask the AI to invent the village again or reroll resource placement.

What a save contains

- **Identity and compatibility:** save version, world ID, generator version, asset-pack version, and creation time.
- **World snapshot:** accepted lore/specification, seed, resolved manifest, entity IDs, and initial story facts.
- **Player:** position, health, stamina, hunger, inventory, equipment, experience, and upgrades.
- **Changes:** harvested nodes, defeated enemies, opened containers, doors, and placed camp kits.
- **Narrative:** quest states, claimed rewards, relationship flags, conversation summaries, and discovered clues.
- **Time:** world clock and any explicit respawn or growth timers.

Storage design

Keep large static manifests separate from small save-state records in IndexedDB. Treat a manifest as immutable once accepted. Commit each checkpoint update in a transaction and retain a previous valid checkpoint. Provide JSON export/import because browser storage can be cleared or evicted.

Save after quest transitions, meaningful inventory transactions, and safe checkpoints. Validate imported records and their size before use. If a new version cannot load a save safely, show a compatibility message and preserve the original file instead of silently regenerating it.

Reproducibility has limits

A fixed seed and pinned generator can reproduce procedural layout. Exact floating-point physics may differ across platforms, so this is not a promise of deterministic simulation replay. For stable load behavior, store resolved placement and relevant state, then rebuild physics from that data.

Failure scenarios and expected behavior

Failure	Recovery
AI fails during generation	Offer bounded repair or a tested template; preserve previous saves.
Browser closes mid-save	Restore the last committed checkpoint; retain the backup.
Asset fails to load	Retry once, use a compatible placeholder for decoration, or block entry if collision/quest-critical.
Model is unavailable during play	Keep simulation running; use authored dialogue and quest options.

Make the world understandable

Screen / state	Essential information and actions
New world	Lore field, example prompt, supported setting, difficulty, generation mode, seed option, and a clear Generate action.
World interpretation	Short setting summary, expected biome/conflict, unsupported requests, and Confirm or Edit.
Generation	Current stage, completed stages, warnings, cancellation, retry/fallback options. Do not invent a precise ETA.
Exploration HUD	Health, stamina, contextual hunger warning, interaction prompt, active quest hint, and compass.
Dialogue	NPC name, readable text, selectable intents, optional free-text question, request status, and Exit.
Inventory / crafting	Equipment, stack counts, recipe ingredients, required station, and clear transaction feedback.
Journal / pause	Known facts, current objectives, map discoveries, save/export, settings, and accessibility controls.

Proposed default controls

WASD — move
Mouse — look
Shift — sprint
Space — jump
E — interact

Left mouse — attack
Right mouse — block
I / J — inventory / journal
1–4 — quick items
Escape — release pointer / pause

Controls are provisional and should be remappable. In a browser, pointer lock requires player interaction; reacquire it through an explicit Resume action.

Comfort and accessibility

- Adjustable sensitivity, field of view, text size, and master/music/effects volume.
- Disable head bob, camera shake, and motion blur by default or offer clear toggles.
- Pair color with icons or text for damage, rarity, and quest states.
- Show subtitles for essential audio information; avoid putting critical clues only in sound.
- Pause single-player action during menus and conversations in the initial version.

> **AI remains visible where useful:** show when a reply is being generated and provide a scripted alternative. Keep model names, token counts, and debug data in developer tools rather than ordinary gameplay screens.

Build in stages with exit criteria

The following is a planning estimate for one developer with AI coding assistance, part-time work, and reusable assets. Assume roughly 15–25 focused hours per week. Re-estimate after the first two milestones; asset and animation work may extend the schedule.

Milestone	Indicative window	Deliverable and gate
M0 — technical spike	Week 1	Record RAM/VRAM; load a PBR reference scene; benchmark one local model; confirm toolchain and licenses.
M1 — engine foundation	Weeks 2–3	First-person movement, collision, interaction, loading, and pause. Complete a fixed test route without clipping or falling through terrain.
M2 — procedural region	Weeks 4–5	Seeded terrain, village, houses, roads, resources, and navigation. Test multiple seeds for overlap and connectivity.
M3 — AI world planning	Weeks 6–7	Lore input → validated plan → generated region. Invalid responses, cancellation, timeout, and fallback all work.
M4 — action and survival	Weeks 8–10	Sword/block, two enemy archetypes, inventory, five recipes, hunger, and small progression system.
M5 — story and saves	Weeks 11–13	Six NPCs, dialogue, one quest with two endings, checkpoints, and save import/export. Full loop works without live AI.
M6 — realistic presentation	Weeks 14–16	Cohesive asset kit, weapon/character animation, lighting, audio, two interiors, and graphics settings.
M7 — evaluation and polish	Weeks 17–18	Generation benchmark, playtests, performance tuning, fixes, demo build, and technical documentation.

Work already started in earlier milestones continues: reference-scene art begins in M0; meaningful save support starts with stable world IDs; testing begins with each system. M6 integrates and polishes assets rather than introducing visuals from scratch.

Scope checkpoint after M3

If prompt relevance or local inference speed is weak, reduce world complexity and compare a smaller model, stronger model, and template baseline. If rendering is the bottleneck, reduce scene density before adding map size. Do not compensate for failed foundations by adding more features.

> **Deliverable hierarchy:** first a technical prototype; then a complete short game; then better visuals and variety. An initial moving scene may be quick to produce, but it is not evidence that the complete RPG is finished.

Prove the world can be played

Generation benchmark

Create 20 prompts spanning ordinary fantasy settings, dense lore, sparse descriptions, contradictory requests, and unsupported features. Use three fixed seeds per prompt: 60 generation cases per provider/model configuration. Store inputs, accepted specifications, repair counts, validation failures, timings, and versions.

Area	Acceptance evidence
Schema and bounds	Every world offered to the player passes validation. Track first-pass success separately; provisional goal ≥95% before repair/fallback.
Geometry / paths	All published worlds have a safe spawn, valid entrances, and reachable mandatory NPCs, resources, and quest targets. Reject failed layouts.
Lore alignment	Human rubric scores setting, required elements, central conflict, and NPC consistency. Record unsupported details and compromises explicitly.
Combat	No multiple damage from one swing; blocking obeys stamina/angle rules; death and loot trigger once; behavior remains stable at varied frame rates.
Inventory / quests	No negative stacks, duplicate rewards, consumed ingredients on failed craft, or unreachable quest states.
Save/load	Round-trip preserves manifest, inventory, quest ending, harvested nodes, and player state. Old/corrupt saves fail clearly.
Failure handling	Test model offline, timeout, malformed output, cancelled jobs, stale dialogue replies, missing assets, and storage errors.

Automated checks and human playtests

Use unit tests for rules with important invariants; generator property tests for many seeds; integration tests for provider validation and saves; and a browser smoke test for lore entry, loading, movement, and interaction. Avoid tests that only restate implementation details.

Run 5–8 short formative playtests with consent. Ask players to generate a world, craft an item, finish the quest, and reload their save. Observe confusion, combat readability, motion comfort, and whether the resulting world matches the requested story.

Optional academic evaluation

If this becomes thesis material, compare a procedural/template baseline against AI-guided generation using the same asset library, map budgets, and seeds. Report relevance, consistency, playability, latency, and cost. Preserve failures and human rating criteria; do not use only the generating model to judge its own output.

Profile before expanding the world

Use the RTX 4060 / i7-13700H machine as the reference system, with RAM, power mode, available VRAM, browser version, resolution, and model settings recorded. Laptop heat and power limits can change results, so compare repeatable runs.

Metric	Provisional target — not yet measured
Rendering	1080p, medium settings, 60 FPS target. Frame-time median ≤16.7 ms, 95th percentile ≤25 ms, 99th ≤50 ms on a fixed traversal.
Procedural compilation	≤15 seconds after the specification is accepted, excluding first-time downloads and model inference.
Initial AI generation	≤120 seconds total as an initial usability target. Record cold model loading separately; show stages and cancellation.
Dialogue	Useful short reply within 5 seconds preferred. Offer a scripted reply after an 8-second wait or on failure. Benchmark with the game rendering.
Memory / stability	No sustained growth after five generate/load/dispose cycles. Keep combined graphics and inference memory below available capacity with headroom.
Loading	Provisional essential-asset payload ≤150 MB; measure compressed transfer and first-playable time separately from full preload.

Optimization order

1. **Find the limit:** separate CPU simulation, GPU rendering, asset loading, garbage collection, and AI latency with recorded profiles.
2. **Reduce repeated work:** share geometry/materials, cache asset loads, instance compatible vegetation, and merge suitable static modules. Instancing reduces draw calls for repeated meshes. [9]
3. **Control detail:** add distance-based detail, chunk-level visibility, lower shadow distance, and adjustable vegetation density.
4. **Control simulation:** use simple static colliders, a small nearby enemy budget, lower-rate distant NPC updates, and worker-based terrain calculations.
5. **Control memory:** dispose unused GPU resources, release decoded images, compress textures, and unload AI models when necessary.
6. **Retest the same routes:** compare before/after traces and visual quality. Keep a change only when it improves the intended metric without breaking behavior.

> **Fallback hardware profile:** lower resolution, shorter shadows, reduced foliage, and scripted dialogue. Do not sacrifice correctness or quest reachability to hit a frame-rate number.

Spend effort where the prototype needs it

Budget area	Planning assumption
Core software / local hosting	Target $0 in required service fees using open tooling and a local development server. Verify component and model licenses when selected.
Local inference	No per-token provider charge; electricity, downloads, disk space, and hardware usage remain real costs.
Art and audio	Start with free compatible assets. Optional $0–100 total reserve for missing animation or cohesive asset kits; this is a budget proposal, not a price quote.
Optional hosted AI	Start with a hard project cap of $10/month if explicitly enabled. Revisit after measuring actual token use; no automatic overage or fallback billing.
Public hosting	Not required for the local prototype. A public demo needs a separate hosting, access-control, and per-user usage plan.

Cloud estimate: cost = input tokens / 1,000,000 × input rate + output tokens / 1,000,000 × output rate. Include repairs and dialogue; record actual provider charges. Pricing and free-tier availability must be checked when a provider is selected.

Primary risks and responses

Risk	Practical response
Realism exceeds the art budget	Approve a reference scene, use one coherent kit, limit environments and animations.
AI invents incompatible content	Restrict vocabulary to supported assets/templates; validate semantics and explain compromises.
Local model harms game performance	Generate before scene load, cap context, serialize requests, unload/offload, or use scripted dialogue.
Procedural world is unplayable	Placement constraints, final-geometry route checks, bounded retries, tested fallback layouts.
Story breaks after player actions	Explicit quest states, stable IDs, protected key NPCs/items, and idempotent rewards.
Scope grows without a complete loop	Use milestone gates. Defer extra systems until the 20–30 minute adventure is playable.

Privacy and access: local mode keeps lore and dialogue on the local machine by design. For any cloud mode, state what is sent and let the player choose it. Treat user lore as data, cap input/output size, and never expose provider secrets in browser code.

The Hollow Watch

> "An isolated forest village mines blue crystals. The old tower has started calling villagers at night, and something is poisoning the woods."

Generated interpretation: a temperate foothill forest; ten stone-and-timber buildings; a crystal resource site; a ruined tower; a frightened village; and a conflict between survival and stopping corruption.

Game element	Concrete result
Six characters	Elder, smith, herbalist, miner, hunter, and a relative of a missing worker. Each has a distinct goal and knowledge boundary.
Exploration	A safe village route, a dangerous forest path, and an optional clue near the mine. A visible tower provides orientation.
Survival / crafting	Starter food; gather herbs and wood; craft a healing item or torch before travelling.
Combat	A corrupted creature on the approach and a hostile humanoid near the tower, using the two supported archetypes.
Quest	Accept investigation → collect a miner's journal → reach the tower → recover the crystal focus → choose its fate.
Two endings	Destroy the focus: safer forest, reduced mining. Stabilize it: mining continues, with a guarded tower. Both use explicit saved flags.

The first implementation tasks

1. Confirm system RAM, available VRAM, free disk space, and whether the initial deliverable is browser-only.
2. Create the TypeScript client/backend/shared-contract workspace and pin dependency versions.
3. Build the realistic reference scene and measure movement/rendering without AI.
4. Benchmark one small local model on five representative world-planning prompts.
5. Implement a hand-authored manifest that can run this example without AI.
6. Add procedural generation, then plug the AI planner into the same manifest compiler.

> **Immediate success condition:** a player can walk through a small fantasy location, interact with an NPC, fight one enemy, craft one item, finish one objective, and reload the result. This is the foundation for expanding the generator.

Still open: installed RAM, desired deadline, preferred combat difficulty, free-text dialogue priority, and browser versus packaged desktop distribution. The plan can proceed using the stated defaults until these are decided.

Sources and implementation vocabulary

Official sources checked on 10 November 2026. These support tool capabilities and asset terms; the architecture, scope, budgets, schedules, and targets are project proposals. Recheck service terms before implementation.

1. **Three.js — GLTFLoader.** glTF loading and supported asset/compression extensions.
   https://threejs.org/docs/pages/GLTFLoader.html

2. **Three.js — Physics.** Physics integration and fixed-step simulation.
   https://threejs.org/manual/pages/physics.html

3. **Three.js — MeshStandardMaterial.** Metallic/roughness physically based materials.
   https://threejs.org/docs/pages/MeshStandardMaterial.html

4. **Khronos — KTX.** KTX2 and Basis Universal texture distribution.
   https://www.khronos.org/ktx/

5. **Poly Haven — Asset License.** CC0 terms for site assets.
   https://polyhaven.com/license

6. **Ollama — Structured Outputs.** Local JSON-schema output; cloud capability limitation.
   https://docs.ollama.com/capabilities/structured-outputs

7. **Ollama — FAQ.** Memory placement, context, and model lifecycle.
   https://docs.ollama.com/faq

8. **Ollama — Pricing.** Local execution versus metered hosted services.
   https://ollama.com/pricing

9. **Three.js — InstancedMesh.** Draw-call reduction for repeated geometry/materials.
   https://threejs.org/docs/pages/InstancedMesh.html

Short glossary

Procedural generation: code creates content from rules and parameters. **Seed:** the starting value for repeatable random choices. **PBR:** physically based rendering. **GLB:** binary glTF 3D asset. **LOD:** level of detail based on viewing conditions. **Manifest:** resolved description of actual world entities and assets. **VRAM:** GPU memory. **Quantization:** a reduced-precision model representation. **Vertical slice:** a small complete section of the intended experience.